Ders 25 Genişletilmiş Notu: Diziler ve Dinamik Bellek, Insertion Sort, Bağlı Listeler, Matris Çarpımı ve Zor Problemler

Ders 25 Genişletilmiş Notu: Diziler ve Dinamik Bellek, Insertion Sort, Bağlı Listeler, Matris Çarpımı ve Zor Problemler

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Veri Yapıları ve Diziler
- 2. Dizinin Boyutu ve Dinamik Bellek
- 3. Sıralı Diziye Ekleme ve Silme
- 4. Insertion Sort
- 5. Dizilerin İkinci Dezavantajı: Ardışık Yer Zorunluluğu
- 6. Bağlı Listeler (Linked List)
- 7. Dizilerle Bağlı Liste Oluşturmak
- 8. Dizilerle Bağlı Liste Kodunun İncelenmesi
- 9. Çift Yönlü ve Dairesel Bağlı Listeler
- 10. Düğüm Yapısı ve Bağlı Listeden Türeyen Veri Yapıları
- 11. Matris Çarpımı
- 12. Matris Çarpım Sırası
- 13. Belleğe Sığmayan Veriyi Sıralamak: External Merge Sort
- 14. Alt Küme Toplamı Problemi
- 15. Gezgin Satıcı Problemi ve Sezgisel Yöntemler
- 16. Projede Algoritma Analizi Nasıl Yapılır?

1. Veri Yapıları ve Diziler

Bu derste kod yazmak yerine ağırlıklı olarak teorik konular işlendi. Hoca, dönem başında kısmen değinilen veri yapıları ve veri soyutlaması (data abstraction) kavramını hatırlatarak başladı.

Soru: Bir mühendis ya da programcı olarak veri yapılarına neden ihtiyaç duyuyoruz? **Cevap:** Veriyi ihtiyaca göre en uygun şekilde depolamak için. Depoladığımız şeye tekrar erişeceğimiz için erişimin de hızlı olması gerekir.

Bazı durumlarda üç temel algoritmik yapı, yani sıralı, şartlı ve tekrarlı yapı, programlama için tam çözüm olmaz. Programlama dilleri bunlara ek olarak belleği organize etmek için çözümler önerir. Bunların en basiti dizidir.

1.1 Çok Boyutlu Diziler ve Bellek

Dizi bir, iki, üç ya da istenen kadar boyutlu olabilir. Hoca `int A[3][4][5]` örneğini, beş tane 3×4 matrisin üst üste konmuş hâli gibi düşünmemizi istedi. `A[3][4]` ise 4 elemanlı 3 dizinin peş peşe sıralanmış hâlidir.

> **Ders sonrası not:** Hocanın anlatımı boyutları gözde canlandırmak için bir benzetmedir. Büyük olasılıkla C'nin bellekteki gerçek düzenini anlatmaz: C'de son indis en hızlı değişir, yani `int A[3][4][5]` bellekte 3 tane 4×5 matris gibi durur. gcc ile yapılan denemede `A[0][0][1]`, `A[0][1][0]` ve `A[1][0][0]` elemanları ilk elemandan sırasıyla 1, 5 ve 20 eleman uzakta çıktı.

Dizinin adı, boyut sayısı ne olursa olsun ilk elemanın yerini gösterir. Tek boyutlu dizide bu `A[0]`, iki boyutlu dizide `A[0][0]`, üç boyutlu dizide `A[0][0][0]` elemanının adresidir.

İki boyutlu bir dizide bellekteki sıra önce ilk satırın elemanlarıyla ilerler: `B[0][0]`, `B[0][1]`, `B[0][2]`. Ardından bir sonraki satır gelir: `B[1][0]` ve devamı.

Programda gördüğümüz dizi 2, 3 ya da 4 boyutlu olabilir. Belleğe baktığımızda ise yapı tek boyutludur, çünkü belleğin kendisi tek boyutludur. Dizinin bu özelliğine ardışıklık denir. Hocanın ifadesiyle dizi bellekte ardışık olarak yerleşir (sequentially located in the memory).

> **Ek bilgi (derste anlatılmadı):** Hocanın sözünü ettiği adres hesabı, büyük olasılıkla şu biçimdedir. S sütunlu iki boyutlu bir dizide `A[i][j]` elemanının adresi taban $+$ $(i \cdot S + j) \cdot$ sizeof(tip) ile bulunur. Bu yüzden erişim süresi indisin büyüklüğüne bağlı olmaz.

1.2 Rastgele ve Sıralı Erişim

Elimizde dizinin adı varken `A[i][j]` elemanına doğrudan erişebiliriz. Bu **rastgele erişimdir** (random access). Örneğin i yerine 6, j yerine 18 verirsek program bir adres hesabı yapar ve ilgili adrese doğrudan gider.

Sıralı erişimde (sequential access) ise ilk elemandan başlanır ve adres birer birer ilerletilir. Pointer'a bir eklemek, tipin boyutu (`sizeof`) kadar eklemek demektir: tip 4 byte ise adres 4 artar, 1 byte ise 1 artar. Bir yerine döngüde değişen i eklenirse elemanlar sırayla gezilir. Sıralı erişim, verileri sırayla girerken ya da okurken işe yarar. Hocanın örneği, 100 kişinin notunu sırayla girmek ve sonra sırayla okumaktı.

2. Dizinin Boyutu ve Dinamik Bellek

Erişimden sonra hoca boyut konusuna geçti. Diziyi yazarken bir boyut belirtmek zorundayız. Bu boyut ya ön işlemci makrosuyla (`#define`) derleme öncesinde verilir ya da 10, 15, 20, 100 gibi sabit bir sayı olarak yazılır. Yani boyut genellikle statiktir.

Oysa okuyacağımız verinin boyutunu program çalışmadan her zaman bilemeyiz. Müzik okuyan bir medya oynatıcı yazılımı buna örnektir. Hoca ayrıca önceki derste bir görüntüyü okuyan kodu tekrar açtı. Kodda satır ve sütun sayısı olan `n` ve `m` önceden tanımlı değildi. Program çalışınca, resmin başlık bilgisini okuyan fonksiyon resimdeki satır ve sütun sayısını, renk derinliğini de nicemleme seviyesinden (quantization level) `Q` değişkenine aktarıyordu.

Bu yüzden dinamik bir mekanizmaya ihtiyaç vardır: dinamik bellek yönetimi (dynamic memory allocation). Hocaya göre önce `scanf` ile `n` okuyup ardından `int D[n]` yazmak mümkün değildir, ama bu mantığa benzer bir yapı vardır.

> **Ders sonrası not:** Hoca böyle bir tanımın olmadığını söyledi. Büyük olasılıkla bu, C89 için doğrudur. C99'daki değişken uzunluklu diziler bu yazıma izin verir, ama büyük olasılıkla büyük boyutlar için yine güvenli değildir.

Bilgisayar bir çip de olabilir, koca bir makine de. Her durumda belleği yöneten işletim sistemidir, dolayısıyla bellekten yer ancak onun izniyle alınabilir. Hoca bunu boyutla da ilişkilendirdi: 500×500'lük bir resim, şimdiye kadar yazılan programlardaki 100-200 elemanlı dizilerden çok daha büyüktür. Küçük boyutlar statik bellek yönetimiyle halledilebilir. Hem boyut belirsiz hem de çok büyükse özel izin gerekir.

> **Ders sonrası not:** Hoca, dinamik belleğin çok yer kapladığı için özel izin gerektirdiğini söyledi. Büyük olasılıkla asıl fark yerin nereden alındığıdır: yerel diziler küçük olan yığında (stack), `malloc` ile alınan yer ise çok daha büyük olabilen öbekte (heap) durur. Boyut da çalışma anında belirlenebilir.

Bu iş için dört temel fonksiyon vardır ve gelecek dönem Yapısal Programlama dersinde görülecektir:

- `malloc`: bellekten yer ayırır (memory allocation).
- `calloc`: yer ayırır ve temizler (clear allocation).
- `realloc`: yeniden yer ayırır (reallocation).
- `free`: yeri serbest bırakır.

Dinamik de olsa statik de olsa, dizi bellekte yine ardışık yerleşir.

3. Sıralı Diziye Ekleme ve Silme

Hoca, dizi kullanmanın algoritmik bir maliyeti olup olmadığını sormak için sıralı diziye ekleme problemini ele aldı. Elimizde küçükten büyüğe sıralı `4 6 8 12` dizisi ve iki boş yer olsun.

3.1 Sona Ekleme

Soru: Bu diziye 13 eklemek için nasıl bir yöntem mantıklıdır? **Cevap:** Yeni eleman için boşluk açmak gerekir, bunun için yer aranır ve ondan büyük elemanlar sağa kaydırılır. Dizi sıralı olduğundan elemanlara tek tek bakılır: 13, sırasıyla 4, 6, 8 ve 12'den büyüktür. Sağda da yer olduğu için sona yazılır.

Hoca bunu hızlandıran daha basit bir yöntem önerdi: Dizinin son elemanının indisi her eklemeden sonra saklanır. Eklemeden önce yeni eleman, bu indisteki elemanla karşılaştırılır. Örneğin 13 eklendikten sonra 18 eklenecekse önce son elemana bakılır. 18 daha büyük olduğu için hemen arkasına yazılır ve son indis bir artırılır.

3.2 Araya Ekleme ve Kaydırma

Eklenecek eleman 13 değil de 7 ise durum değişir. Bir döngü elemanıyla baştan itibaren yer aranır: 4 ve 6'nın yeri uymaz, 8'e gelince yer bulunmuş olur. Bu yerin indisi i'de saklanır. En son elemanın indisi de j olsun. Sonra sondan başlanarak i'ye kadar bütün elemanlar birer sağa kaydırılır (shift).

`4 6 8 12` dizisine 7 ekleme adımları şöyledir:

1. Baştan arama yapılır ve $i = 2$ bulunur, çünkü 8 ilk kez 7'den büyüktür.
2. Son indis yedeklenir, çünkü kaydırma sırasında j'nin değeri bozulur: $tmp = j = 3$.
3. $j \geq i$ olduğu sürece `A[j+1] = A[j]` yapılır ve j bir azaltılır. Önce 12 sağa geçer, sonra 8 sağa geçer.
4. j, i'nin altına düşünce döngü biter ve `A[i] = 7` yazılır.
5. Son indis $tmp + 1 = 4$ olur. Dizi artık `4 6 7 8 12` şeklindedir.

```c
#include <stdio.h>

int main(void)
{
    int A[6] = {4, 6, 8, 12};
    int son = 3, x = 7, i = 0, j, tmp;

    while (A[i] < x)
        i++;
    j = son;
    tmp = j;
    while (j >= i) {
        A[j + 1] = A[j];
        j--;
    }
    A[i] = x;
    son = tmp + 1;
    for (i = 0; i <= son; i++)
        printf("%d ", A[i]);
```

```
    printf("\n");
    return 0;
}
```

Bir öğrenci, j'nin sıfıra kadar inmesini içeren başka bir yazım önerdi. Hoca sıfıra kadar gitmeye gerek olmadığını söyledi: Baştaki elemanlar değişmeyecek, yalnızca bulunan yerden sona kadarki kısım kaydırılacaktır.

3.3 Kaydırmanın Maliyeti

Kaydırma döngüsü j ile i arasındaki fark kadar çalışır. Bu fark n ise n işlem yapılır, yani karmaşıklık $O(n)$ olur. Tek bir eleman eklemek için n işlem yapmak oldukça maliyetlidir. Elemanı sona ekleyip diziyi yeniden sıralamak ise bildiğimiz sıralama algoritmalarıyla n^2 tutardı.

Aynı sorun silmede de vardır. 6'yı çıkarmak istersek sağındaki elemanların birer sola kayması gerekir. Sonuçta dizide hem ekleme hem de çıkarma n işlem gerektirir.

4. Insertion Sort

Insertion sort'un mantığını günlük hayatta da kullanırız. Elimizde oyun kartları olsun ve bunları birden ona kadar diziyor olalım. Bir kartı alır, diğerleri arasında yerini bulur ve araya koyarız. "Insert" zaten araya yerleştirmek demektir.

4.1 Bubble Sort ile Karşılaştırma

Bubble sort yan yana iki elemana bakar, sıra bozuksa yer değiştirir. Hoca `4 3 2` kısmı üzerinden bunun gereksiz takaslar içerdiğini gösterdi. Önce 4 ile 3, sonra 4 ile 2, en sonunda 3 ile 2 takas edilir: `3 4 2`, `3 2 4`, `2 3 4`. Oysa doğrudan 4 ile 2'yi takas etmek daha mantıklı görünür. Hoca insertion sort'un gereksiz takasları iyileştirdiğini söyledi. Gereksiz kontrolleri ise pek azaltmadığını kendisi ekledi.

> **Ders sonrası not:** Hocanın "4 ile 2'yi doğrudan takas etmek" önerisi insertion sort'un yaptığı iş değildir. Altyazıdaki izlemeye göre insertion sort önce `3 4 2`, sonra `2 3 4` üretir ve elemanları kaydırır. `4 3 2` için python denemesinde kaydırma sayısı da bubble sort'un takas sayısı da 3 çıktı. Büyük olasılıkla kazanç, sayının azalmasından değil, bir kaydırmanın tek atama, bir takasın ise üç atama olmasından gelir.

4.2 Algoritmanın Mantığı

Algoritma ilk elemandan değil, ikinci elemandan başlar ve önceki elemanlara bakar. Örnek dizi `4 3 2 10 12` olsun. Hoca sonra diziye önce 1, sonra 5 ekledi.

1. **3 için:** 3, 4'ten küçüktür, 4 sağa kayar ve 3 başa geçer: `3 4 2 10 12`.
2. **2 için:** 2, önce 4 ile sonra 3 ile karşılaştırılır. Kendinden küçük bir eleman görene kadar aramaya devam eder. Başa yerleşir, 3 ve 4 birer sağa kayar: `2 3 4 10 12`.
3. **10 için:** Öncekilerden küçük değildir, yerinde kalır.
4. **12 için:** Öncekilerden küçük değildir, yerinde kalır.
5. **1 için:** Hepsinden küçüktür, en başa geçer: `1 2 3 4 10 12`.
6. **5 için:** Sondan bakılır: 12'den küçük, 10'dan küçük, 4'ten küçük değil. Burada durur ve 4 ile 10 arasına girer: `1 2 3 4 5 10 12`.

Her adımda şu garanti vardır: O ana kadar bakılan elemanların hepsi kendi içinde sıralıdır.

4.3 Kod

Hoca kodu kâğıt üzerinde sözde kod gibi yazdı. Yerin ne zaman bulunacağı belli olmadığı için iç döngüde `while` kullanılır. Aşağıda aynı algoritma, dersin örnek dizisiyle çalışan bir C programı olarak verilmiştir.

```
#include <stdio.h>

void insertionSort(int arr[], int n)
```

```
{
    int i, j, key;

    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

int main(void)
{
    int a[] = {4, 3, 2, 10, 12, 1, 5};
    int n = 7, i;

    insertionSort(a, n);
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Programın çıktısı `1 2 3 4 5 10 12` olur.

`i = 1` ikinci elemandan başlamayı sağlar. `key` değişkeni o anki elemanı yedekler, çünkü kaydırma sırasında üzerine yazılacaktır. `j = i - 1` ile elemanın kendisine değil, bir önceki elemana bakılır: i 1 ise 0'a, 2 ise 1 ve 0'a, 3 ise 2, 1 ve 0'a bakılır. j en fazla 0'a kadar inebilir, bu yüzden koşulda `j >= 0` vardır.

4.4 Adım Adım İzleme

Hoca kodu `4 3 2 10` üzerinde izledi.

Birinci tur ($i = 1$):

1. `key = 3`, `j = 0` olur.
2. $j \geq 0$ doğrudur ve `arr[0] = 4 > 3` olduğu için `arr[1] = 4` yazılır, j eksi 1 olur.
3. Döngüden çıkılır. `arr[j+1]`, yani `arr[0]`, 3 olur.
4. Dizi `3 4 2 10` olur.

İkinci tur ($i = 2$):

1. `key = 2`, `j = 1` olur.
2. `arr[1] = 4 > 2` olduğu için `arr[2] = 4` yazılır, $j = 0$ olur.
3. `arr[0] = 3 > 2` olduğu için `arr[1] = 3` yazılır, j eksi 1 olur.
4. Döngüden çıkılır ve `arr[0] = 2` yazılır.
5. Dizi `2 3 4 10` olur.

Sonraki turlarda 10 yerinde kaldığı için dizi değişmez.

5. Dizilerin İkinci Dezavantajı: Ardışık Yer Zorunluluğu

Hoca dizilerin iki dezavantajı olduğunu söyledi. Birincisi kaydırma işlemidir. İkincisi de elemanların bellekte ardışık yerleşme zorunluluğudur. Ardışıklık normalde bir avantajdır, çünkü birinin adresini biliyorsak toplama yaparak diğerine geçer, iki döngüyle bütün matrisi gezeriz.

Soru: Peş peşe gelmenin kaydırma dışında ne gibi bir dezavantajı olabilir? **Cevap:** Öğrenciler önce kaydırma maliyetini ve eleman sayısını önerdi. Hoca birincisinin zaten sayıldığını, ikincisinin de sorun olmadığını söyledi. Asıl cevap, yeterince bitişik yer bulunamamasıdır.

Diyelim ki 1 milyon hücreye ihtiyacımız var. Bellekte de toplam 1,5 milyon boş hücre (free cell) bulunuyor, ama bu hücreler bitişik değil (not contiguous). Boş yerlerin içinde bitişik 1 milyonluk bir parça varsa dizi ayrılabilir. Veriler dağınıksa, örneğin bir yerde 500 binlik, başka yerlerde 300 binlik, 200 binlik ve 400 binlik parçalar varsa, 1 milyonluk yer alınamaz. Yer statik de istense dinamik de istense sonuç aynıdır, çünkü bu yapılar doğası gereği bitişik olmak zorundadır.

> **Ders sonrası not:** Hocanın "ne bileyim" diyerek saydığı parçaların toplamı 1,4 milyondur. Toplam 1,5 milyona tamamlanması için kalan 100 binin başka parçalarda olduğu düşünülebilir. Sonuç değişmez, çünkü en büyük parça 500 bindir.

6. Bağlı Listeler (Linked List)

Çözüm yeni bir veri yapısıdır: bağlı liste ya da bağlantılı liste (linked list). Bellekten biraz fazla yer vererek daha fazla yeri yönetebilmeyi amaçlar. Hem kaydırma sorununu ortadan kaldırır hem de belleğin bazı durumlarda daha etkili kullanılmasını sağlar. Eleman sayısı da baştan sabitlenmez, 300 ya da 400 eleman fark etmeksizin istenildiği kadar ekleme yapılabilir.

Temel fikir şudur: Bir bellek hücresinin bir kısmında veri saklanır. Ek bir alanda da bir sonraki hücrenin adresi tutulur.

6.1 Haber Zinciri Benzetmesi

Hoca bir senaryo kurdu: Size sınav saatini duyuracak, ama herkese birden mesaj gönderemiyor. E-posta da yok, Telegram da yok. O zaman haberi bir öğrenciye gönderir, o öğrenci bir sonrakine gönderir, o da bir sonrakine, böyle devam eder. En sondaki öğrenci kimseye göndermez. Her öğrenci bir kişiden mesaj alır ve sonuncu hariç bir kişiye gönderir. Hoca ise yalnızca gönderir, almaz.

Bu yapı bir tren katarına benzer ve bağlı listenin kendisidir. Her birim bir düğümdür (node). Haberi ortaya atan ilk elemana **head** denir. Son elemanın kendisinden sonra göndereceği kimse olmadığı için adres alanına boş değer, yani `NULL` yazılır. Bu haliyle yapı tek yönlüdür.

6.2 Bellekteki Görünüm

Düğümlerin bellekte ardışık olması gerekmez. Her düğümde herhangi bir tipte veri ve sonrakinin yerini gösteren bir alan vardır. Son düğümün bu alanında `NULL` bulunur.

Hocanın hazırladığı ve yayımlanacağını söylediği dokümandaki hayali örnekte düğümler şu adreslerdedir:

Adres	Veri	Sonraki adres
100	1	108
108	2	200
200	3	194
…	…	…
250	$n-1$	40
40	n	`NULL`

Adreslerin artarak gitmesi gerekmez, 200'den sonra 194'e geri dönülebilir.

Veri alanı yalnızca temel bir tip olmak zorunda değildir. Bir yapı (structure), örneğin bir polinomun bilgileri, de saklanabilir.

Soru: Bütün listeyi gezmek için neyi bilmemiz gerekir? **Cevap:** İlk elemanın adresini, yani head'i. Bu örnekte 100'ü.

Soru: Listenin sonuna geldiğimizi nasıl anlarız? **Cevap:** Baktığımız sonraki adres `NULL` ise son elemana gelmişizdir.

6.3 Erişim ve Ekleme

Bağlı listede bir hücreden sonrakine pointer aritmetiğiyle, yani toplama yaparak geçilemez. Sonraki düğüme ancak adres alanı üzerinden gidilir. Rastgele erişim söz konusu değildir. Aradaki bir yere ulaşmak için head'i bilmek ve baştan başlayıp n adıma kadar ilerlemek gerekir.

> **Ders sonrası not:** Hoca bir yerde bağlı listede "sıralı erişim aslında yok" dedi. Doğrusu rastgele erişimin olmadığı, erişimin sıralı olduğudur. Hoca bunu bir öğrencinin sorusu üzerine dersin ilerisinde "random access olmaması ve sequential olması" diye kendisi söylüyor. O cümlede kastettiği, elemanların bellekte yan yana olmamasıdır.

Bellek dolana kadar yeni yer ayrılabilir. Araya eleman eklemek de kolaydır: Yeni düğüm, araya gireceği düğümün sonrasını gösterir. Önceki düğüm de yeni düğümü gösterecek şekilde değiştirilir ve eski bağlantı kopar. Hiçbir elemanın yeri değişmez.

6.4 Dizi ile Bağlı Liste Arasındaki Ödünleşim

Hoca burada "bu kelimeyi bizden çok duydunuz" diyerek ödünleşim (trade-off) kavramını hatırlattı.

Özellik	Dizi	Bağlı liste
Ek bellek	Yok, doğrudan veri saklanır	Her düğümde fazladan `next` alanı
Erişim	Rastgele erişim var	Baştan sona gezmek gerekir
Ekleme ve çıkarma	Maliyetli	Daha kolay
Yer	Bitişik olmak zorunda	Bitişik olması gerekmez
Uzunluk	Bellekte toplam kapasite olsa bile bitişik yer yoksa istenen kadar alınamaz	Dinamik olarak değişir

7. Dizilerle Bağlı Liste Oluşturmak

Şimdiye kadarki bilgilerle bağlı liste dizilerle de oluşturulabilir. Hoca, normalde bağlı listelerin böyle değil, yapılarla (structure) kurulduğunu ve bunun gelecek dönem görüleceğini vurguladı. Burada mantığı görmek için daha basit bir yol izlenir.

7.1 Başlangıç Durumu

Veri dizisi `4 3 2 1 5` olsun. İkinci bir **link** dizisi, `next` alanının yerini tutar. Bir de head için normal bir tam sayı değişkeni vardır: `head = 3`. Link dizisinde eksi 1, listenin sonu yani `NULL` anlamındadır.

İndis	Veri	Link
0	4	4
1	3	0
2	2	1
3	1	2
4	5	-1

Liste head'den izlenir: 3. hücrede 1 var ve 2'yi gösteriyor. 2. hücrede 2 var ve 1'i gösteriyor. 1. hücrede 3 var ve 0'ı gösteriyor. 0. hücrede 4 var ve 4'ü gösteriyor. 4. hücrede 5 var ve link eksi 1 olduğu için liste bitiyor. Veriler bellekte karışık dursa da sırayla `1 2 3 4 5` okunur.

7.2 Araya Ekleme

Dizide 6. bir yer olduğunu varsayalım ve 2 ile 3 arasına 8 ekleyelim. Sıralı olması gerekmiyor, senaryo böyle.

1. Yeni eleman, mevcutları bozmadan dizinin boş yerine yazılır: `veri[5] = 8`. Kaydırma yapılmaz.
2. Head'den başlanır ve 2 aranır. 2'yi bulmak yeterlidir, çünkü 3 ondan hemen sonra gelir. 3. hücrede 1 vardır, aranan bu değildir, sonraki hücreye gidilir. 2. hücrede 2 vardır ve yer bulunmuş olur.
3. 2. hücrenin linki yedeklenir: `tmp = link[2] = 1`.
4. 2. hücrenin linkine 8'in adresi yazılır: `link[2] = 5`.
5. 8'in linkine yedeklenen değer yazılır: `link[5] = tmp = 1`.

İndis	Veri	Link
0	4	4
1	3	0
2	2	**5**
3	1	2
4	5	-1
5	**8**	**1**

Yeni sıra `1 2 8 3 4 5` olur. Listenin sonu yine eksi 1'dir. Veri dizisine bir, link dizisine iki olmak üzere üç yazma yapıldı ve n defa kaydırma gerekmedi. Bir öğrencinin isteği üzerine hoca bu adımları bir kez daha tekrar etti.

7.3 Silme

Şimdi 4'ü silelim. Bağlı listede arama kesinlikle baştan yapılır. Ayrıca silinecek elemanın bir öncesi ile bir sonrası akılda tutulmalıdır: 4'e kimden geldik, 4'ten kime gideceğiz? Hoca bunlara x ve y dedi. Bu değerler head'den itibaren sürekli güncellenir.

1. Head'den başlanır: 3. hücredeki 1, 2. hücredeki 2, 5. hücredeki 8 ve 1. hücredeki 3, aranan değer değildir.
2. 0. hücrede 4 bulunur. Bir önceki hücre $x = 1$, bir sonraki hücre ise `link[0]` değeri olan $y = 4$ olur.
3. Önceki düğüm artık 4'ün sonrasını gösterir: `link[1] = 4`. Eski değer olan 0 silinmiş olur.

Yeni sıra `1 2 8 3 5` olur. 0. hücrede veri ve link değerleri durmaya devam eder, ama hiçbir düğüm orayı göstermediği için bağlı listede yer almaz.

Hoca, burada aslında bellekten silme yapılmadığını da belirtti. Dizide 5 elemanlık yer vardır ve bir hücreyi sıfırlamak ya da eksi 1 yapmak yer kazandırmaz. Hücre bellekte yer kaplamaya devam eder. Yine de boş olduğunu belirtmek için linkine özel bir değer yazılabilir. 0 yazılamaz, çünkü 0 geçerli bir indistir. Eksi 1 listenin sonu olarak kullanıldığı için eksi 2 gibi bir değer seçilebilir. Hoca kendi yaklaşımını şöyle anlattı: Yeni eleman eklerken dizi taranır ve yer yoksa linki eksi 2 olan hücreye yazılır, sonra linkler bağlanır.

> **Ek bilgi (derste anlatılmadı):** Boşalan hücreler büyük olasılıkla kendi aralarında ikinci bir listede de tutulabilir. Buna boş hücre listesi (free list) denir. Bu durumda yeni eleman eklerken boş hücreyi bulmak için diziyi taramaya büyük olasılıkla gerek kalmaz.

7.4 Dokümandaki Örnek: Head'in 0. İndiste Tutulması

Hocanın dokümanındaki örnekte, bir `A` dizisi ile sonraki elemanları gösteren bir `PO` dizisi tek yönlü bir liste oluşturur. Burada ayrı bir head değişkeni yoktur, `A`'nın 0. hücresi boş bırakılır ve `PO[0]` head görevini görür. İki dizi de normal tam sayı dizisidir.

İndis	A	PO
0	(head)	5
1	2	3
2	5	4
3	4	2

İndis	A	PO
4	8	-1
5	1	1

Sıra: `PO[0]` 5'i gösterir ve okunan değerler `1 2 4 5 8` olur.

7'yi sıralı ekleme. Gösterilen değer 7'den küçükse bir sonrakine geçilir, büyükse durulur.

1. 5. hücrede 1 vardır, 7'den küçüktür, 1. hücreye geçilir.
2. 1. hücrede 2 vardır, 7'den küçüktür, 3. hücreye geçilir.
3. 3. hücrede 4 vardır, 7'den küçüktür, 2. hücreye geçilir.
4. 2. hücrede 5 vardır, 7'den küçüktür, 4. hücreye geçilir.
5. 4. hücrede 8 vardır, 7'den küçük değildir, durulur.
6. Yeni eleman boş olan 6. hücreye yazılır: `A[6] = 7`.
7. 5'in bulunduğu hücre yeni elemanı gösterir: `PO[2] = 6`.
8. 7, 8'in hücresini gösterir: `PO[6] = 4`.

Yeni sıra `1 2 4 5 7 8` olur.

Soru: Bu eklemede kaç işlem yapıldı? **Cevap:** Kaydırma yapılmadı, atama yapıldı. Bir atama elemanı dizinin sonuna yazdı, iki atama da linkleri güncelledi. Toplam üç atama yapıldı. Bu, sabit zamanlı yani $O(1)$ bir işlemdir.

Sıralı ekleme yapılacaksa, eklenecek yere kadar yine arama gerekir, bu da n adıma kadar sürebilir. Hocaya göre normal dizide de arama yapılacağı için aramalar aynıdır, ama dizide aramadan sonra gelen kaydırma maliyeti burada yoktur. Bunun karşılığında n eleman için n tane de link elemanı gerekir. Bu, bellek ile işlem karmaşıklığı arasındaki bir ödünleşimdir.

> **Ders sonrası not:** Hoca iki yapıda aramanın aynı olduğunu söyledi. Eleman sıralı dizide doğrusal aranıyorsa bu doğrudur. Sıralı bir dizide büyük olasılıkla ikili arama (binary search) da kullanılabilir, bağlı listede ise rastgele erişim olmadığı için büyük olasılıkla kullanılamaz.
>
> **Hocanın vurgusu:** Bellek ile işlem karmaşıklığı arasındaki bu tür ödünleşimleri hep göreceksiniz.

4'ü silme. Bu yeni durumda head yine 0. indistir.

1. `PO[0] = 5` olduğundan 5. hücreye gidilir. Burada 1 var ve 4'ten farklı. 1. hücreye geçilir.
2. 1. hücrede 2 var ve 4'ten farklı. 3. hücreye geçilir.
3. 3. hücrede 4 var, bulundu. Geçici değişkenlerde önceki hücre 1, sonraki hücre `PO[3] = 2` tutulur.
4. `PO[1] = 2` yazılır. 1. hücre artık 3'ü değil, 2'yi gösterir.

Yeni sıra `1 2 5 7 8` olur. `A[3]` ve `PO[3]` değerlerinin dizide durması bağlı liste açısından bir anlam ifade etmez. Hoca, bir öğrencinin bu konudaki önceki sorusuna atıf yaparak bu hücrelerin boşta durduğunu söyledi ve `PO[3]` değerini eksi 2 yapmayı önerdi. Eksi 1 listenin sonunu, eksi 2 ise boş bir yeri gösterir.

8. Dizilerle Bağlı Liste Kodunun İncelenmesi

Hoca bu konuyu ekranda bir kod üzerinden gösterdi ve bunun normal bir bağlı liste değil, dizilerle oluşturulmuş bir bağlı liste olduğunu tekrar etti. Kodda `A`, `B` ve `PO` adlı üç dizi vardır. Elemanlar önce `B` dizisine karışık biçimde girilir. Burada 10 elemanlı yer açılmıştır, kullanılan n daha küçüktür. `PO[0]` head olarak kullanıldığı için veriler indis 1'den başlar.

Aşağıdaki program hocanın anlattığı akıştan kurulmuştur.

```c
#include <stdio.h>
#define MAX 10

void elemanEkle(int A[], int i, int ek, int PO[])
{
    int k = 0;
```

```
    int j = PO[0];

    A[i] = ek;
    while (j != -1 && A[j] < ek) {
        k = j;
        j = PO[j];
    }
    PO[k] = i;
    PO[i] = j;
}

void elemanSil(int A[], int sil, int PO[])
{
    int k = 0;
    int j = PO[0];

    while (j != -1 && A[j] != sil) {
        k = j;
        j = PO[j];
    }
    if (j == -1)
        printf("Eleman yok\n");
    else
        PO[k] = PO[j];
}

void siraliYazdir(int A[], int PO[])
{
    int j = PO[0];

    while (j != -1) {
        printf("%d: veri=%d link=%d\n", j, A[j], PO[j]);
        j = PO[j];
    }
}

int main(void)
{
    int A[MAX] = {0}, B[MAX] = {0}, PO[MAX] = {0};
    int n, i, ek, sil;

    scanf("%d", &n);
    for (i = 1; i <= n; i++)
        scanf("%d", &B[i]);

    PO[0] = 1;
    A[1] = B[1];
    PO[1] = -1;
    for (i = 2; i <= n; i++) {
        ek = B[i];
        elemanEkle(A, i, ek, PO);
    }
    siraliYazdir(A, PO);

    scanf("%d", &ek);
    elemanEkle(A, n + 1, ek, PO);
    siraliYazdir(A, PO);

    scanf("%d", &sil);
    elemanSil(A, sil, PO);
```

```
    siraliYazdir(A, PO);
    return 0;
}
```

8.1 Eleman Ekleme Fonksiyonu

Program önce `PO[0] = 1` yazar, ilk veriyi `A[1]`'e alır ve `PO[1] = -1` yapar. Liste şimdilik tek elemanlıdır. Sonra ikinci elemandan başlanır: `ek = B[i]` alınır ve `elemanEkle` çağrılır. Link dizisi fonksiyona dizi olarak gittiği için fonksiyondaki değişiklik çağıran tarafa da yansır.

Fonksiyon önce değeri `A[i]`'ye yazar. Değer hep bu hücrede kalır, yeri değişmez. Sıralama yalnızca `PO` üzerinde yapılır. `k` önceki düğümü, `j` geçerli düğümü tutar. Döngü, `j` eksi 1 olmadığı ve `A[j]` eklenecek değerden küçük olduğu sürece ilerler. Döngü bitince `PO[k] = i` ile önceki düğüm yeni elemanı, `PO[i] = j` ile yeni eleman sonraki düğümü gösterir.

Hoca tahtada `B` dizisini `4 5 2 1` aldı. `B[1] = 5` ilk veri oldu. `i = 2` için `ek = 2` fonksiyona gitti.

> **Ders sonrası not:** Hoca bu adımda `j`'nin 0 olduğunu söyledi ve döngüye girildiğini varsayarak `PO[1] = 2`, `PO[2] = -1` sonucuna ulaştı. Doğrusu farklıdır. `j = PO[0] = 1` ile başlanır. `A[1] = 5 < 2` yanlış olduğu için döngüye girilmez, `k` 0 olarak kalır. Sonuç `PO[0] = 2` ve `PO[2] = 1` olmalıdır, yani 2 listenin başına geçer. Hocanın sonucu küçükten büyüğe sıra vermezdi. Aynı şekilde `i = 3` için `ek = 1` eklenince `PO[0] = 3` ve `PO[3] = 2` olur, sıra `1 2 5` çıkar. `B[0]`'daki 4 ise indis 0 head'e ayrıldığı için bu kodda listeye girmez. Hocanın kendi çalıştırdığı örnekte de ilk hücre 0 görünüyor.

8.2 Silme ve Yazdırma

Silmede de aranan eleman bulunana kadar `PO[0]`'dan başlanarak gezilir. Eksi 1 görülürse döngüden çıkılır ve eleman yok demektir. Eleman bulunursa önceki düğüm sonraki düğümü gösterecek şekilde atama yapılır.

Sıralı yazdırmada head elemanından başlanır. `j` eksi 1'den farklı olduğu sürece `j`, `A[j]` ve `PO[j]` yazılır, sonra `j = PO[j]` ile bir sonraki linke geçilir.

Hoca, kodu kendi yazıp bir kez izlemenin, tahtadaki işlemlerin kod hâlini anlamayı kolaylaştıracağını söyledi.

8.3 Çalıştırma Örneği

Hoca programı `n = 4` ve `3 6 2 1` değerleriyle çalıştırdı. Dizi `0 3 6 2 1` oldu. En küçük eleman 1, 4. indistedir, dolayısıyla `PO[0] = 4` olur. Sonra 2 (3. indis), 3 (1. indis) ve 6 (2. indis) gelir.

İndis	A	PO
0	0	4
1	3	2
2	6	-1
3	2	1
4	1	3

Soru: Listeye 5 eklenirse `A` dizisindeki yeri ve `PO` dizisi ne olur? **Cevap:** 5, `A`'da 5. indise yazılır. Head 4'ü gösterir: 1 küçüktür, 3. hücreye geçilir. 2 küçüktür, 1. hücreye geçilir. 3 küçüktür, 2. hücreye geçilir. 6 büyüktür, durulur. `PO[1]` artık 2 yerine 5 olur, `PO[5]` de 6'nın adresi olan 2 olur. Bir öğrencinin verdiği `4 5 -1 1 3 2` cevabı doğrudur.

Bir silme denemesinde ekrana araya giren sıralı yazdırma çıktıları kafa karıştırdı, hoca bunları kapatarak programı yeniden çalıştırdı. Son denemede `A` dizisi `0 2 6 4 3 5` oldu ve 4 silindi. Head 1'i gösterir, 1'den 4'e, 4'ten 3'e gidilir. 3. hücrede 4 vardır ve bu silinecek değerdir. Önceki hücre olan 4. hücrenin linki 3 yerine 5 olur, yani doğrudan bir sonrakini gösterir. Hoca izleme sırasında bir an "5'i sileceksem" dedi, ardından silinenin 4 olduğunu düzeltti. Programda hata yoktur.

> **Hocanın vurgusu:** Bağlı listeler veri yapıları dersinde daha ayrıntılı görülecek. Bu ders kapsamında bu kodu incelemeniz yeterlidir.

9. Çift Yönlü ve Dairesel Bağlı Listeler

9.1 Çift Yönlü Bağlı Liste

Şimdiye kadar yalnızca ileri gidildi. Düğüme ikinci bir işaretçi eklenirse hem sonraki (`next`) hem önceki (`previous`) düğüm tutulabilir. Hoca bunu önce mesajlaşma üzerinden anlattı: Hoca bir öğrenciye bir şey söyleyebilir, o öğrenci de hocaya söyleyebilir, kendinden sonrakine de söyleyebilir.

Hiyerarşik işler bu yapıya iyi bir örnektir. Bölüme bir dilekçe verildiğinde dilekçe önce fakülteye, oradan öğrenci işleri daire başkanlığına, oradan ilgili birime gider. Bölüm doğrudan bir başkasına gönderemez. Kabul ya da ret cevabı da aynı yoldan önce daire başkanlığına, sonra fakülteye, sonra bölüme ve en son dilekçe sahibine döner.

Bellekte önceki örnek şöyle olur: 100 adresli düğümün önceki alanı boştur, çünkü ilk elemandır, sonraki alanı 108'dir. 108'in önceki alanı 100, sonraki alanı 200'dür. 200'ün önceki alanı 108, sonraki alanı 194'tür.

Listenin tümünü gezmek dizide de bağlı listede de n adım sürer. Bellek açısından ise link için tek yönlü listede n, çift yönlü listede $2n$ yer gerekir. Doğrudan erişim çift yönlü listede de yoktur: Silme ya da değiştirme yapılacaksa o yere head'den başlanarak ulaşılmalıdır.

9.2 Dairesel Bağlı Liste

Soru: Tek yönlü listede son elemanın `NULL` alanına ne yazılırsa liste dairesel olur? **Cevap:** İlk elemanın adresi. Örnekte 100 yazılır ve listenin sonundan tekrar başa gelinir.

Soru: Çift yönlü listede ne yazılır? **Cevap:** Son elemanın sonraki alanına ilk düğümün adresi olan 100, ilk elemanın önceki alanına son düğümün adresi olan 40 yazılır.

Gerçek programda adreslerin kaçta oturduğunu bilemeyiz. Onun yerine pointer kullanılır. `tmp` en son ulaşılan düğüm ise `tmp->next = head` ile liste başa döner.

> **Ders sonrası not:** Hoca çift yönlü durumda `head`'in önceki alanına `tmp`'nin `next` değerinin yazılacağını söyledi. Doğrusu `head->prev = tmp` olmalıdır. `tmp->next = head` yapıldıktan sonra `head->prev = tmp->next` yazılırsa gcc ile yapılan denemede görüldüğü gibi `head` kendini gösterir.

10. Düğüm Yapısı ve Bağlı Listeden Türeyen Veri Yapıları

Dizilerle yapılan örnekler bağlı liste gibi görünse de hâlâ dizidir. n eleman için ardışık $2n$ yer kullanıldı. Örneğin 10 eleman için hem veriye hem linke 10 elemanlı bir dizi gerekir. Bu kadar bitişik yer yoksa liste kurulamaz. Ardışıklık sorunu sürer.

Asıl çözüm, pointer ya da nesne yönelimli programlamadaki referans mantığıyla kendi veri tipimizi tasarlamaktır. Düğümde herhangi tipte bir veri ve sonraki düğümün adresi saklanır. C'de bu alan düğüm türünden olamaz, düğüme pointer türünden olur:

```
struct node {
    int data;
    struct node *next;
};

struct dnode {
    int data;
    struct dnode *next;
    struct dnode *prev;
};
```

Çift yönlü liste için ikinci yapıdaki gibi `next` ve önceki alanı olmak üzere iki pointer ile üç alan kullanılır. Hoca Java, C# ve C++'ta düğümün referans türünde elde edilebildiği için alanın

doğrudan düğüm türünde yazılabildiğini söyledi. Veri alanı `int`, karakter ya da örneğin `book` türünde bir sınıf olabilir.

> **Ders sonrası not:** Bu ifade büyük olasılıkla Java ve C# için geçerlidir. C++'ta sınıf türünden bir alan nesnenin kendisini içerdiği için, büyük olasılıkla C'de olduğu gibi yine pointer gerekir: `Node *next;`.

Şimdiye kadarki yapı özetle şöyledir: Bir head adres tutar. Her düğümde veri ve sonraki düğümün adresi vardır. Son düğümün adres alanı `NULL`'dır. İki yönlü yapıda düğümde önceki ve sonraki düğüm de bulunur.

Bağlı listeleri anlatmanın iki nedeni var: Birincisi algoritmik faydalarıdır. İkincisi, onlardan türeyen veri yapılarıdır. Yığın (stack), kuyruk (queue) gibi dinamik yapılar, ağaçlar (tree) ve graflar bağlı listeleri kullanır. Örneğin ağaçta iki ya da üç çocuklu düğümler vardır. Düğümde `next` yerine `right` ve `left` yazılır ve kaç çocuk varsa onların adresi saklanır.

Hoca bunu bellekteki bir örnekle bağladı. Bellekteki A, B, C, D verilerini kendimiz organize ederiz: A, B'nin adresini; B, D'nin adresini; D, C'nin adresini; C de A'nın adresini gösterir. Böylece veriler hiç kopyalanmadan birbirine bağlanır. Özellikle nesne yönelimli dillerde referans mekanizması ve verilerin referansla aktarımı, gereksiz kopyalamadan, zaman ve bellek kaybından kurtarır.

11. Matris Çarpımı

Sıradaki slayttaki kumarbaz oyunu gibi olasılıksal konular şimdilik geçildi. Matris çarpımı sayısal analizde de görüldüğü için kısaca üzerinde duruldu.

11.1 Boyut Kuralı ve Sezgi

Önce boyutlar kontrol edilir. A matrisi $M \times N$, B matrisi $X \times Y$ boyutlu olsun. Çarpım için $N = X$ olmalıdır. Sonuç olan C ise $M \times Y$ boyutlu olur.

Bu eşitliğin nedeni, birinci matrisin bir satırı ile ikinci matrisin bir sütununun karşılıklı çarpılabilmesidir. Karşılıklı hücreler çarpılır ve çarpımlar toplanır. Bu bir iç çarpımdır (dot product). Örneğin sonucun birinci satır, birinci sütun hücresi şöyle bulunur:

$$c_{11} = a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} + ...$$

> **Ders sonrası not:** Hoca burada "cross product, dot product'ın bir uygulaması" dedi. Anlatımındaki işlem karşılıklı çarpıp toplamaktır, yani iç çarpımdır. Dış çarpımla (cross product) büyük olasılıkla bir ilgisi yoktur.

11.2 Algoritma

Bir hücreyi bulmak için bir döngü gerekir. Sonraki sütunları doldurmak için bir döngü, sonraki satırları doldurmak için bir döngü daha gerekir. Yani toplam üç döngü vardır. Ara değişken `sum` hücrenin toplamını biriktirir. Hoca son hâlinde A'yı $n \times m$, B'yi $m \times p$ aldı, böylece C de $n \times p$ olur.

```
for i = 1 .. n
    for j = 1 .. p
        sum = 0
        for k = 1 .. m
            sum += A[i][k] * B[k][j]
        C[i][j] = sum
```

Üç iç içe toplam vardır ve iç döngüde bir temel işlem yapılır. Karmaşıklık $O(n \cdot p \cdot m)$ olur. Kare matrislerde $n = m = p$ olduğundan bu $O(n^3)$ demektir.

11.3 Neden Önemli?

Hoca bu problem üzerinde neden bu kadar çalışıldığını sordu. Bilgisayar grafiği, oyun programlama, robotik, görüntü işleme gibi pek çok mühendislik alanında matris çarpımı önemli bir işlemdir. Denklem sistemi çözümleri gibi bilimsel uygulamalarda da karşımıza çıkar.

Hocanın örneği robot koludur: Kolunuzu bir robot kolu gibi düşünün. Parmak ucunun x, y, z koordinatlarını bildiğimizi kabul edelim. Kolu dirsekten sağa ya da sola çevirince parmak ucunun konumu da değişir. Yeni konum robotikte matris çarpımıyla hesaplanır. Kolu 90 derece döndürmek gibi eksen dönüşümleri de böyledir. Bu işlem ne kadar az maliyetle yapılırsa o kadar iyidir.

Karmaşıklık daha da indirilebilir. Hoca üssün yaklaşık $2{,}80$ civarına inebildiğini söyledi: $O(n^{2{,}8})$.

> **Ek bilgi (derste anlatılmadı):** Hocanın sözünü ettiği algoritma büyük olasılıkla Strassen algoritmasıdır. Üssü büyük olasılıkla $\log_2 7 \approx 2{,}807$ değerinden gelir.

12. Matris Çarpım Sırası

Birden fazla matris art arda çarpılacaksa bir iyileştirme yapılabilir. Hocanın örneği: A 10×30, B 30×5, C 5×60 boyutlu. $(AB)C$ ve $A(BC)$ aynı sonucu verir, ama işlem sayıları farklıdır.

$(AB)C$ **için:**

1. A ile B çarpılır: $10 \cdot 30 \cdot 5 = 1500$ çarpma. Ara sonuç 10×5 olur (ara adım eklendi).
2. Bu ara sonuç C ile çarpılır: $10 \cdot 5 \cdot 60 = 3000$ çarpma.
3. Toplam $1500 + 3000 = 4500$ çarpma.

$A(BC)$ **için:**

1. B ile C çarpılır: $30 \cdot 5 \cdot 60 = 9000$ çarpma. Ara sonuç 30×60 olur (ara adım eklendi).
2. A bu ara sonuçla çarpılır: $10 \cdot 30 \cdot 60 = 18000$ çarpma.
3. Toplam $9000 + 18000 = 27000$ çarpma.

Sonuç aynı olduğu hâlde biri 4500, öteki 27000 işlemdir. Demek ki çarpım sırası optimize edilebilir. Bunun analizi ileride veri yapıları ya da algoritma analizi derslerinde görülecektir.

13. Belleğe Sığmayan Veriyi Sıralamak: External Merge Sort

Merge sort diziyi ikiye bölerek en küçük hâline getirir: 8'i 4'e, 4'ü 2'ye, 2'yi 1'e böler. Sonra parçaları birleştirir. Bu işlem sırasında kendisi ayrıca n elemanlık bir belleğe ihtiyaç duyar. Bellekte yeterli yer yoksa external merge sort kullanılır.

Örnek: 9 GB verimiz ve 1 GB RAM'imiz var.

Soru: Parçaları kendi içinde sıralarken merge sort kullanabilir miyiz? **Cevap:** Hayır. Merge sort, 1 GB veri için ayrıca 1 GB yer ister. Bizde ise toplam 1 GB vardır. Bir öğrenci "binary sort" önerdi. Hoca öyle bir sıralamanın olmadığını, adının ikili arama (binary search) olduğunu söyledi. Bubble sort n^2 olduğu için pahalıdır. Quicksort $n \log n$ olduğu için kullanılabilir.

Algoritma şu adımlarla ilerler:

1. Veri RAM'e sığacak parçalara bölünür: 9 GB, 1 GB'lık 9 parçaya ayrılır.
2. Her parça sırayla RAM'e alınır, quicksort ile sıralanır ve diske geri yazılır. Bu durumda elimizde her biri kendi içinde sıralı 1 GB'lık 9 parça vardır, ama verinin tamamı sıralı değildir.
3. Her parçanın başından yaklaşık %10'luk bir kısım, yani 100 MB çekilir. $9 \cdot 100 = 900$ MB yaklaşık 1 GB'lık yerde tutulur.
4. Daha önce görülen iki sıralı diziyi birleştirme mantığıyla bu 9 alt dizi birleştirilir. İlk elemanlara bakılır, en küçüğü alınır.
5. Birleştirilen kısım diske yazılır. Parçalardan yeni %10'luk bölümler çekilir ve işlem bütün veri bitene kadar sürer.

Buradaki “external” sözü, veriyi diskten parça parça RAM’e çekip işlemek ve diske geri yazmak anlamındadır. Hoca bir yerde “belleğe yazıyoruz” dedi, ama sonraki cümlesinde bunun diske geri yazma olduğunu açıkça belirtti.

> **Ders sonrası not:** Hoca bu kısımda hem %10 hem de %11 dedi. 100 MB ve 900 MB hesabı %10’a karşılık gelir. %11 ise 1 GB’ın dokuzda birine yakındır, bu durumda RAM tamamen dolardı.
>
> **Ek bilgi (derste anlatılmadı):** Geriye kalan yaklaşık 100 MB’lık yer büyük olasılıkla çıkış tamponu olarak kullanılır. Birleştirilen sıralı veri burada birikir ve tampon dolunca diske yazılır.

14. Alt Küme Toplamı Problemi

Problem şudur: Bir sayı dizisinde toplamı k olan bir alt küme var mı? Bu problemin benzeri laboratuvarda özyinelemeli olarak yapılmıştı.

Hocanın örnek dizisi `4 8 3 6 5 7`, hedef toplam ise 10’dur. Alt küme bir, iki, üç, dört elemanlı ve en fazla n elemanlı olabilir. Önce tek elemanlı alt kümelere birer birer bakılır. Sonra iki elemanlı alt kümelere: 4 ile birlikte 8, 3, 6, 5 ve 7 denenir, sonra 8 ile birlikte diğerleri denenir. Bu, iç içe iki döngü demektir. Üç elemanlı alt kümeler için i, j, k olmak üzere üç, dört elemanlılar için dört döngü gerekir.

Boş küme dikkate alınmazsa toplam $2^n - 1$ olasılık vardır. Hocanın örneğinde bu, $n = 6$ için $2^6 - 1 = 63$ alt kümedir (ara adım eklendi). Karmaşıklık 2^n mertebesindedir.

> **Ders sonrası not:** Hoca önce 2^n dedi, sonra iç içe döngü sayısının artmasından yola çıkarak problemin karmaşıklığının $n!$ mertebesinde olduğunu söyledi. Doğrusu, kendi sayımının verdiği 2^n’dir. $n = 6$ için $6! = 720$ olurdu, oysa denenecek alt küme sayısı 63’tür.

Hoca bu problemin özyinelemeli çözümle, alt problemlere bölünerek biraz daha hızlı bulunabileceğini ekledi. Optimum çözüm, gerçekten bu toplamı sağlayan alt kümedir. Hoca örnek olarak $6 + 4 = 10$ ve $7 + 3 = 10$ çözümlerini söyledi. Python ile bütün alt kümeler denendiğinde, bu dizide toplamı 10 olan yalnızca bu iki alt küme bulundu. Hocanın andığı $5 + 4 + 1$ ise yalnızca dizide 1 olsaydı geçerli olacak bir örnektir.

15. Gezgin Satıcı Problemi ve Sezgisel Yöntemler

15.1 Problem

Elimizde noktalar ve bunların arasındaki bağlantılardan oluşan bir graf vardır. Bir noktadan başlayıp kalan bütün noktalara birer kez uğrayarak başlangıca geri dönmek istiyoruz. Hoca bunun aslında bir Hamilton devresi (Hamiltonian circuit) problemi olduğunu söyledi ve bir şekil üzerinde, bir noktadan başlayıp aynı noktaya dönen bir yol gösterdi. Birden fazla çözüm varsa her birinin maliyetine, örneğin aradaki kilometrelere, bakmak gerekir. Gezgin satıcı problemi bununla ilgilidir.

> **Ders sonrası not:** Hoca çizimde “aynı yoldan iki defa geçmiyorum” dedi. Büyük olasılıkla bu ölçüt Euler devresine aittir. Hamilton devresinde koşul, her **noktaya** bir kez uğramaktır.

Soru: n şehir için kaç farklı durum vardır? **Cevap:** $n!$. İlk şehir için n, sonra $n-1$, sonra $n-2$ seçenek vardır, en sonda bir şehir kalır ve başlangıca dönülür.

> **Ders sonrası not:** Başlangıç şehri sabitlenir ve rotanın iki yönü aynı sayılırsa farklı rota sayısı büyük olasılıkla $\frac{(n-1)!}{2}$ olur. Bu sayı da büyük olasılıkla kaba kuvvet için hâlâ çok büyüktür.

Türkiye’deki 81 il için $81!$ çok zor bir problemdir. Bu tür optimizasyon problemlerinde bütün olasılıkları tek tek denemek, yani kaba kuvvet (brute force), işin içinden çıkılamayacak kadar uzun sürer. Bunun için sezgisel (heuristic) algoritmalar ve açgözlü (greedy) tasarım teknikleri vardır.

15.2 Açgözlü Yaklaşım

Soru: Bu problemi sezgisel ya da açgözlü yöntemle nasıl çözersiniz? **Cevap:** Öğrenciler önce her şeyi denemeyi önerdi. Hoca bunun kaba kuvvet olduğunu, hiçbir açgözlü yöntemin her şeyi denemeyeceğini söyledi. Rastgele seçim önerisi de geldi. Hoca, rastgele seçimin körü körüne bir arama (blind search) olduğunu belirtti. Sezgisel ya da açgözlü bir teknik ise belli bir bilgiye dayanarak en iyi çözüme götürmeyi hedefler, ama garanti etmez.

Hocanın yöntemi en yakın komşuya gitmektir. Ankara'yı seçelim. Oradan Eskişehir'e, Kırşehir'e, Konya'ya ya da Aksaray'a gidilebilir. Bunlardan en yakını hangisiyse ona gideriz. Sonra gittiğimiz şehirden yine en yakın şehre geçeriz. Başka seçeneklerde belki ileride daha küçük bir mesafe bulunacaktı, ama onları göz ardı ediyoruz. Ankara'dan Konya'ya gidildiyse Ankara'dan Eskişehir'e ya da Kırşehir'e artık gidilmez. En sonda bir yerden Ankara'ya dönülür.

Her adımda gidilebilecek şehir sayısı azalır: $n-1$, sonra $n-2$, sonra $n-3$ ve en sonda 1. Toplam n^2 mertebesindedir ve $n!$'den çok çok küçüktür.

> **Ders sonrası not:** Hoca toplamı $\frac{n(n+1)}{2}$ olarak söyledi. Saydığı terimler $n-1$'den 1'e kadar olduğu için doğrusu $\frac{n(n-1)}{2}$'dir. Mertebe yine n^2'dir.

Tek sorun, bulunan çözümün optimum olmamasıdır. Bu, yaklaşık (approximate) ya da uygulanabilir (feasible) bir çözümdür. Yine de körü körüne bir yöntemden çok daha iyidir. Hocanın körü körüne örneği plaka sırasıyla gezmekti: Adana'dan başla, Adıyaman'a git, oradan üçüncü plakaya geç. Bu çok uzun sürer.

> **Ders sonrası not:** Hoca üçüncü plakanın Amasya olup olmadığını sordu. Büyük olasılıkla 03 Afyonkarahisar'dır, Amasya ise 05'tir.

15.3 Optimum Çözümün Garanti Olmadığına Dair Örnek

Hoca slayttaki bir grafı kullandı. Anlatımdan çıkarılan kenarlar şöyledir (ara adım eklendi):

```
A --2-- B --1-- C --1-- D --1-- E --1-- F
A ------------------6------------------ F
```

Rota	Hesap	Uzunluk
B, C, D, E, F, A (açgözlü)	$1+1+1+1+6$	10
B, A, B, C, D, E, F	$2+2+1+1+1+1$	8
A, B, C, D, E, F	$2+1+1+1+1$	6

B'den başlayan açgözlü yöntem önce en yakın olan C'ye gider, sonra D, E, F üzerinden 6'lık kenarla A'ya ulaşır. Oysa B'den A'ya gidip geri dönmek toplam 4 tutar, öbür taraftan A'ya gitmek ise 6 tutar. Bu yüzden B, A, B, C, D, E, F rotası 8 ile daha kısadır. Başlangıç A olsaydı rota 6 olurdu.

> **Ders sonrası not:** Hoca açgözlü rotada "6 giderim, 2 giderim" dedi, ama toplamı 10 olarak verdi. Kendi saydığı son 2'lik dönüş eklenirse toplam 12 olur. Tablodaki 10, 8 ve 6 başlangıca dönmeyen rotalardır. Ayrıca B, A, B rotası B'ye iki kez uğradığı için problemin "birer kez" koşuluna uymaz. Hocanın göstermek istediği sonuç değişmez: En yakın komşu seçimi en iyi rotayı garanti etmez ve sonuç başlangıç noktasına bağlıdır.

Hocaya göre her sezgisel yaklaşım sonucu garanti etmez. Bazıları, bazı problemlerde ve kısıtlar altında optimal çözümü garanti eder, ama çoğunlukla böyle değildir.

15.4 Yapay Zekâ Arama Algoritmaları

Karınca kolonisi optimizasyonu, parçacık sürü optimizasyonu, arı algoritması gibi yapay zekâ arama algoritmaları ve ileride görülecek kapsayan ağaç (spanning tree) algoritmaları bu tür problemlerde, özellikle yarışma problemlerinde kullanılır. Karşılaştırma ölçütleri sürenin kısalığı (örneğin 3 dakikada bitmesi), daha az bellek harcanması ve bulunan rotanın kısalığıdır.

Optimumu bulmanın şart olmadığı durumlarda çok beklemek yerine optimal olmayan ama hızlı bulunan çözümler tercih edilir. Hocanın sözüyle hiç çözümsüzlük olmaktansa en kötü çözüm iyi bir çözümdür. Bir gezgin satıcıya "5 yıl bekle, bütün olasılıkları deneyip söyleyeyim" denmez. Önce bir sonuç bulunur, daha iyisi bulunursa çözüm güncellenir. Sezgisel optimizasyonların çoğu böyle çalışır.

16. Projede Algoritma Analizi Nasıl Yapılır?

Bir öğrencinin sorusu üzerine hoca, yapısal programlama projesinde karmaşıklığın nasıl incelenmesi gerektiğini anlattı. Karmaşıklık iki türlü incelenir: çalışma zamanı ve bellek.

16.1 Çalışma Zamanı: Sayaç ve Tablo

Program size n^3 ya da n^4 gibi bir ifade söylemez. Ama örneğin $n = 100$ girdiğinizde işlem sayısını elde edebilirsiniz. Bunun için bir sayaç (counter) kullanılır. Döngüde en çok yapılan işlemin, yani temel işlemin (basic operation), bulunduğu yerde sayaç bir artırılır. Bu temel işlem örneğin karşılaştırma olabilir. Değişik n değerleri için elde edilen işlem sayıları bir tabloya yazılır.

İstenirse sonuç yıldızlarla basit bir histogram olarak çizilir. Ortak bir kat seçilir ve her değer bu kata bölünerek yıldız sayısı bulunur. Hocanın örneğinde $T(n)$ değerleri n^2 biçiminde ilerler ve kat 25'tir:

n	$T(n)$	Yıldız sayısı
10	100	$100/25 = 4$
15	225	$225/25 = 9$
20	400	$400/25 = 16$

```
n = 10 | ****
n = 15 | *********
n = 20 | ****************
```

> **Ders sonrası not:** Hoca 15 için önce 255 dedi, sonra "225 miydi" diye sordu. Doğrusu $15^2 = 225$'tir. Yıldız sayısı olan 9 da 225'e karşılık gelir. Dersin devamında 255 değeri bir kez daha geçti.

16.2 Zaman Ölçmek Neden Yetmez?

Soru: Bir öğrenci projesinde bir menü yaptığını söyledi: $n = 10$, $n = 1000$ ve $n = 4000$ için ayrı ayrı çalışıp süreyi veriyor, süre de artıyor. Bu olur mu? **Cevap:** Menü sorun değildir, ama hocanın kastettiği saat ölçümü (`clock`) değildir. $T(n)$, temel işlemin kaç kez yapıldığıdır. Yani saniye değil, işlem sayısıdır. Bu yüzden sayaç kullanılmalıdır.

`clock` ölçümü de ayrıca alınabilir, ama deterministik bir ölçü değildir, çünkü bilgisayarda o anda başka birçok program da çalışır. Yine de $T(n)$ ile süre arasında bir orantı görülmesi beklenir. Hocanın örneği: 100 işlem 10 saniye sürüyorsa, 400 işlemin 40 saniye civarında sürmesi gerekir. n, $T(n)$'yi etkiler. $T(n)$ de süreyle orantılıdır.

> **Ders sonrası not:** Hoca aynı örnekte 255 işlem için 25 saniye dedi. Aynı orantıda 225 işlem için 22,5 saniye beklenir.
>
> **Hocanın vurgusu:** Asıl iş zaman ölçümü değildir. Zaman göreceli bir kavramdır: Sizin bilgisayarınızda n^2 çalışan bir algoritma benim bilgisayarımda kısa sürebilir, $n \log n$ çalışan bir algoritma da başka bir bilgisayarda daha uzun sürebilir. Algoritma analizinden kastımız temel adım sayısıdır (number of basic steps). Dersin amacı da budur: Tasarlanmış bir algoritmayı analiz etmek.

Dersin önceki haftalarında iteratif algoritmaların analizinde toplam sembolleriyle yapılan işlemler, bir sayaçla program içinde rahatça hesaplanabilir.

16.3 Temel İşlem Nedir?

Bir sıralama algoritmasında önce kontrol, sonra takas yapılır. Temel işlem olarak takas alınamaz, kontrol alınır. Her algoritmada o algoritmanın temel işlemine bakılmalıdır.

16.4 Diffie-Hellman Örneği

Soru: Bir öğrencinin projesinde Diffie-Hellman algoritması, yani mod alma ve üs alma var. Bunlar kısa sürüyor, nasıl analiz edilmeli? **Cevap:** Üs alma, mod almadan daha maliyetlidir ve algoritmik yükü çeker. Bu nedenle temel işlem olarak üs alma alınmalıdır.

Öğrenci hazır `pow` fonksiyonunu kullanıyordu. Hoca bunun muhtemelen $\log n$ karmaşıklıkla çalıştığını söyledi: $n = 100$ için yaklaşık 7, $n = 1000$ için yaklaşık 10 eder. Çünkü $\log_2 100 \approx 6{,}64$ ve $\log_2 1000 \approx 9{,}97$ (ara adım eklendi). $T(n)$ değerleri, süreler ve n değerleri birlikte karşılaştırılabilir.

> **Ders sonrası not:** Hoca, matematik kütüphanesindeki üs almanın $\log n$ ile çalıştığını bildiğimizi söyledi. Büyük olasılıkla bu her kütüphane için geçerli bir garanti değildir, çünkü `pow` gerçel sayılarla çalışır ve uygulaması kütüphaneye bağlıdır. Diffie-Hellman'daki büyük tam sayılı üs alma için de büyük olasılıkla uygun değildir.

Bazı algoritmalarda temel işlemleri ve $T(n)$'yi doğrudan hesaplamak mümkün değildir. Hazır bir fonksiyonun nasıl çalıştığı bilinmiyorsa önce araştırılmalı, yazılışına bakılıp analiz edilmelidir. Hazır bir şey kullanmak bu yönden sorunlu olabilir, gerekirse fonksiyon kendiniz yazılabilir. Üs alma için böl ve yönet algoritması görülmemiş olsaydı, `for` döngüsüyle klasik bir algoritma yazılır ve karmaşıklığının $O(n)$ olduğu söylenirdi: 100 için 100 işlem gibi.

16.5 Bellek Karmaşıklığı

Bellek için, programda n'e bağlı olarak oluşturulan dizilere bakılır. Bir yerde $2n$, bir yerde $4n$ elemanlık dizi oluşturuluyorsa bunlar doğrudan toplanır. Bu örnekte toplam $6n$ olur (ara adım eklendi). 100 eleman verildiğinde 600 elemanlık yere ihtiyaç duyulduğu raporlanabilir.

Dersin Toparlaması

Bu derste dizilerin bellekte tek boyutlu ve ardışık durduğu, bunun hızlı rastgele erişim sağladığı, ama ekleme ve silmede n işlemlik kaydırma ile bitişik yer zorunluluğu getirdiği görüldü. Insertion sort bu kaydırma mantığı üzerine kuruludur. Bağlı listeler ise fazladan bir `next` alanı karşılığında bu iki sorunu çözer, ama rastgele erişimi kaybeder. Dizi ile bağlı liste arasındaki seçim, ders boyunca vurgulanan ödünleşimin somut bir örneğidir. Matris çarpımında $O(n^3)$'lük temel algoritma ile çarpım sırasının maliyete etkisi, external merge sort'ta belleğe sığmayan verinin parçalanarak sıralanması gösterildi. Alt küme toplamı ve gezgin satıcı gibi zor problemlerde kaba kuvvetin çok pahalı olduğu, sezgisel ve açgözlü yöntemlerin hızlı ama optimum olması garanti edilmeyen çözümler ürettiği anlatıldı. Son olarak bir algoritmanın saniye ölçülerek değil, temel işlem sayılarak analiz edilmesi gerektiği vurgulandı.

Bağlı listelerin yapı ve pointer'larla gerçek uygulaması ile dinamik bellek fonksiyonları gelecek dönem görülecektir. Bir sonraki derste, önceki konunun ürün üçlüleri kodu, görüntü işleme örneği ve bir oyuncunun oyun kazanma olasılığıyla ilgili olasılıksal algoritma örnekleri ele alınacaktır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.